AI for science

Models that learn the simulator,
and loops that search with a checker

Carlos Cotrini

ETH Zurich

21 October 2026

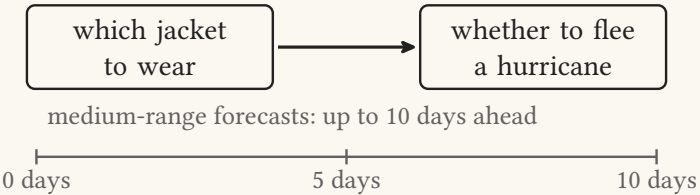
1

Paper 1

GraphCast

Learn the next six hours of weather, then repeat

Every day, decisions wait on the forecast





1. We all use forecasts, from the jacket to the hurricane.

2. Medium-range: the next 10 days.

3. Weather bureaus (ECMWF in Europe, NOAA in the US, the Met Office in the UK) issue them every day.

Lam et al., Science 382, 1416-1421 (2023), https://doi.org/10.1126/science.adi2336

Today a forecast is a physics simulation

the laws of the atmosphere,
solved by computer

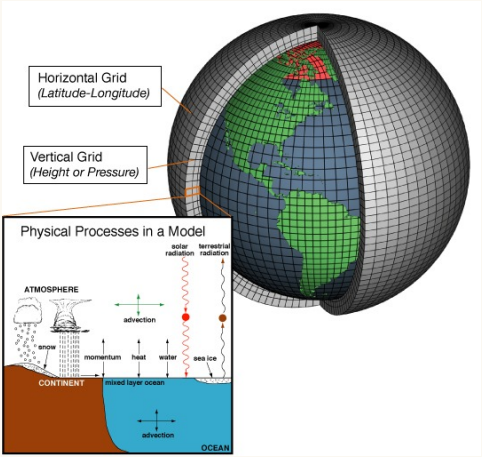
↓

some of the largest
supercomputers in the world

↓

one 10-day forecast: about
an hour on 11,664 cores

better physics comes from
experts by hand; decades
of past weather sit unused



The idea: learn the forecast from past weather



better than the leading physics forecast, called HRES, on 90% of 1,380 checks

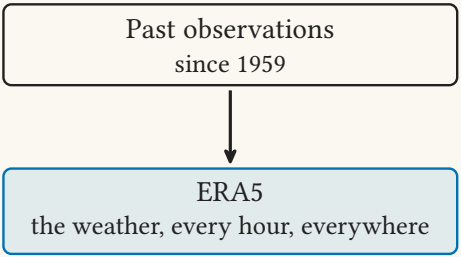
10 days in under a minute, on a single chip

1. A neural network learns from 39 years of past weather.

2. Result: beats HRES, the best physics forecast, on 90% of 1,380 checks.

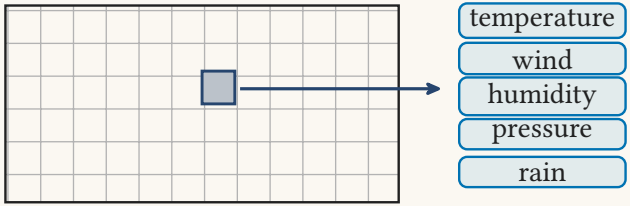
3. Cost: under a minute on one AI chip (a TPU v4).

ERA5: the weather of the past, hour by hour



ERA5 is a dataset of the weather of the past; it makes no forecast.

The weather, written as numbers on a map

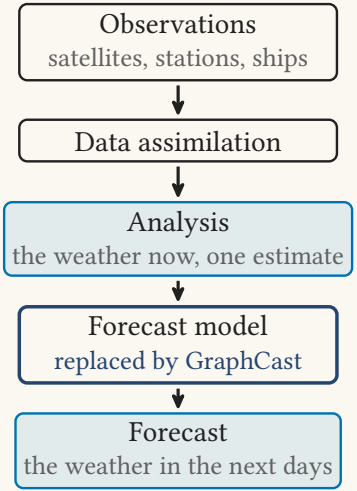


one cell every 0.25 degree



Cut the Earth into a grid of positions, and learn to estimate the weather in the future from the current measurements.

A forecast has two parts, GraphCast replaces one



1. Data assimilation merges the latest observations into one estimate of the weather now: the **analysis**.

2. A **forecast model** rolls the analysis forward. Numerical weather prediction (NWP) solves the equations of the atmosphere on a supercomputer.

3. GraphCast replaces the forecast model only. It learned from past weather how the next hours follow the present.

2

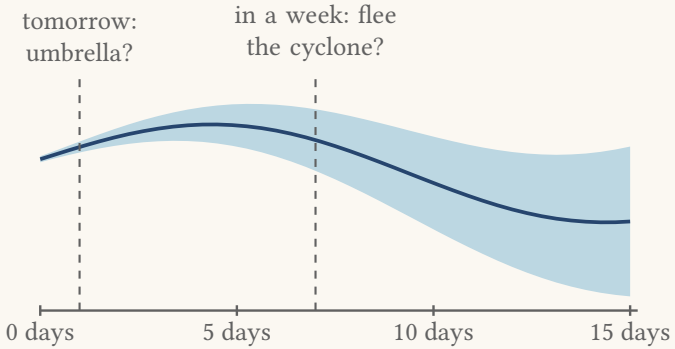
Paper 2

GenCast

Draw many sharp futures instead of one blurry average

A forecast grows less certain with time

Today's weather is known only in part, and small errors grow with time.



1. The line is the forecast: our best guess.

2. The band is the other weather that could happen. It widens with time.

3. Tomorrow the band is thin. A week out, a decision needs the band.

Price et al., Probabilistic weather forecasting with machine learning, Nature 637:84-90 (2025) https://doi.org/10.1038/s41586-024-08252-9*; figures from the paper, CC BY 4.0*

The idea: many sharp samples of the weather

GenCast, a neural network, draws sharp futures; their average is a blur.

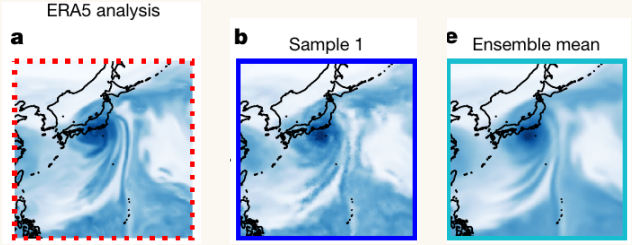
the weather now and before

↓

GenCast

↓

50 forecasts of 15 days



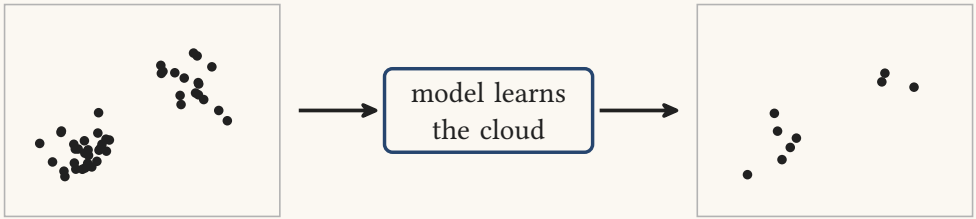
what happened | one GenCast sample | average of 50 samples

better than ENS, ECMWF's 50-forecast ensemble, on 97.2% of 1,320 targets

a target: one variable, at one level, at one lead time

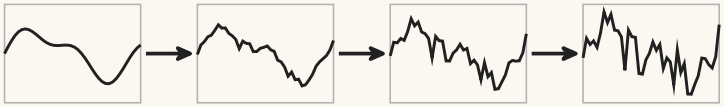
Generative models draw new samples

GenCast is a generative model; first, what such a model does.



Diffusion, part 1: bury data in noise

Diffusion, GenCast's kind of generative model, starts by destroying data.



noisy version = real data + noise of strength σ

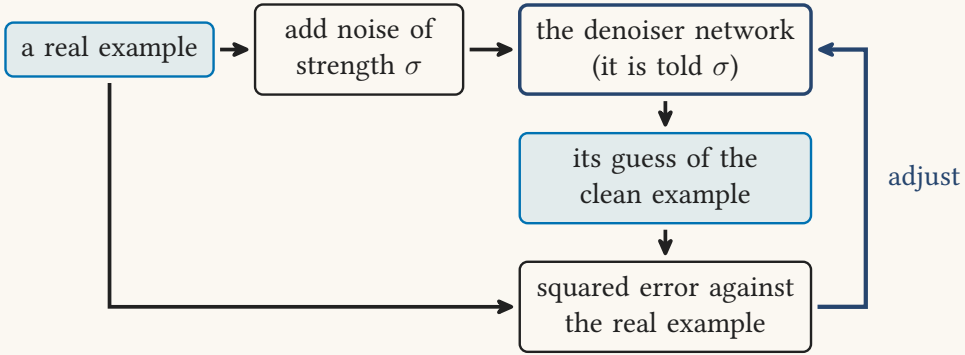
1. Take a real example (here a curve).

2. Add noise, then more, until only noise is left.

3. This half needs no learning: adding noise is easy.

Diffusion, part 2: learn to remove noise

Training: a network learns to clean noisy copies of real data.



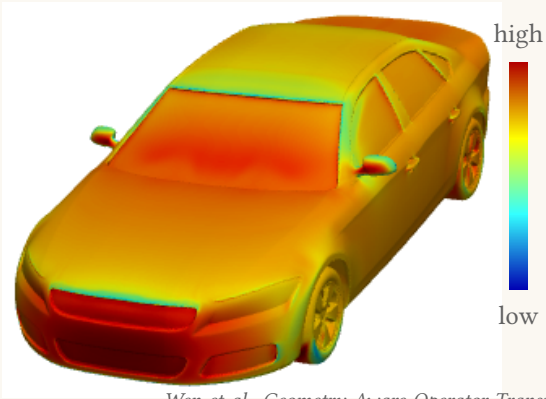
3

Paper 3

GAOT

Teach a network what a simulation solver does

This car is not a photograph

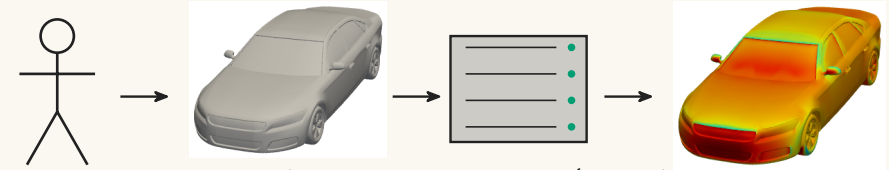


Red: high pressure, the air pushes hard on the car.
Blue: low pressure.

A computed picture. Every new design needs one.

Wen et al., Geometry Aware Operator Transformer as an Efficient and Accurate Neural Surrogate for PDEs on Arbitrary Domains, NeurIPS 2025, papers.nips.cc; figures from the paper (CC BY 4.0)

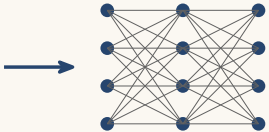
A design is simulated again and again



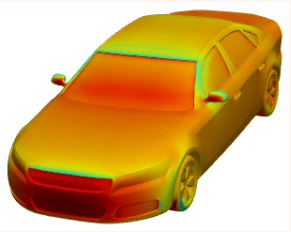
What if the picture took a blink?



the shape

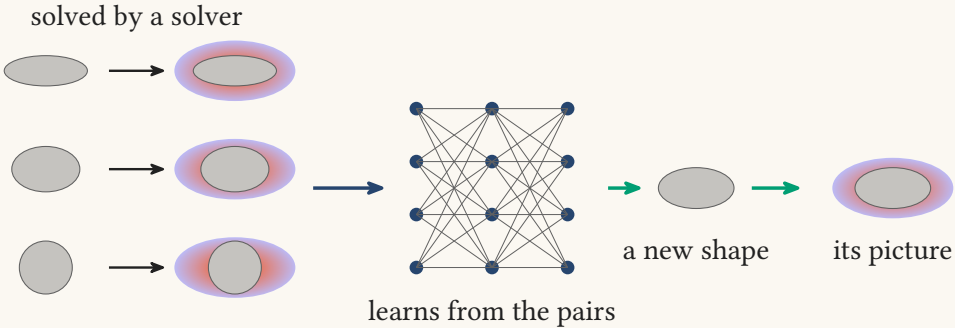


a trained network



the pressure it predicts

Teach a network the solver's answers



The expensive work is done once, when the pairs are made.

4

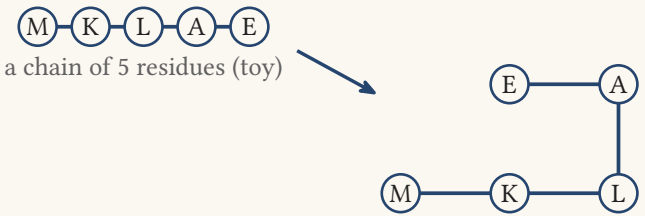
Paper 4

AlphaFold 2

Predict a protein’s shape from its sequence

A protein is a chain that folds into a shape

Proteins are essential to life; their shape helps explain what they do.



1. A protein: a chain of amino acids, 20 kinds, written as letters.

2. The chain folds into one 3D shape: its structure.

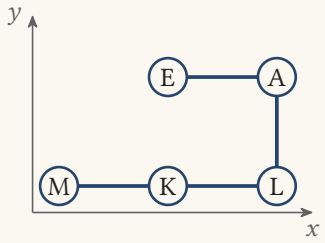
3. An open problem for over 50 years: predict the shape from the letters.

Each amino acid of the chain is called a residue.

Jumper et al., Nature 596 (2021), p. 583; SI 1.2.9; the toy chain: ours

A structure is a table of atom positions

As data, a structure is three coordinates for every atom, in ångström.



residue	letter	x	y	z
1	M	0	0	0
2	K	4	0	0
3	L	8	0	0
4	A	8	4	0
5	E	4	4	0

1. One point per residue here: its central carbon.

2. The structure: a table of coordinates in ångström (Å).

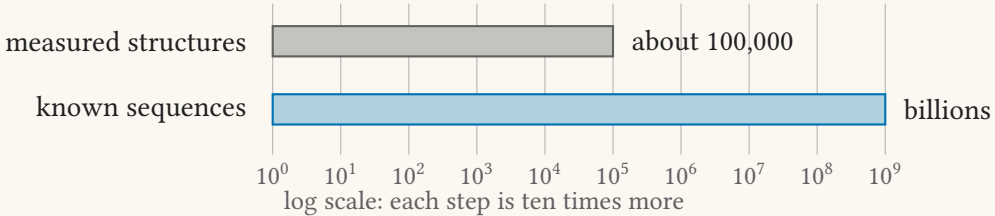
3. 1 Å = 0.1 nm; a carbon atom is about 1.4 Å wide.

The answer to predict: three numbers for every atom of the protein.

Jumper et al., Nature 596 (2021), pp. 584, 585; SI 1.8.1; the toy: ours

Measuring one structure takes months or years

Sequences are known by the billion; each structure takes months to years.



1. About 100,000 unique proteins have a measured structure.

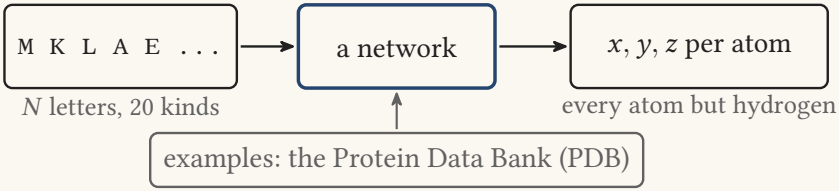
2. Billions of sequences are known; each structure takes months to years.

Only a small fraction of known sequences has a structure: hence prediction.

Jumper et al., Nature 596 (2021), p. 583

The task: sequence in, coordinates out

As machine learning: a string of letters in, a set of 3D points out.



1. Input: the sequence of letters.

2. Output: the coordinates of its atoms.

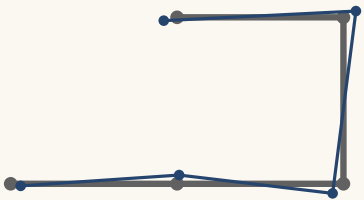
3. Learned from structures that experiments solved.

A supervised problem: sequences in, coordinates out, learned from the PDB.

Jumper et al., Nature 596 (2021), pp. 583, 585; Methods (Training regimen)

The score: r.m.s.d., in ångström

Fit the prediction onto the experiment, then average the gaps between atoms.



grey: experiment
blue: prediction

1. Turn and shift the prediction to fit the experiment best.

2. r.m.s.d.: the root of the mean squared gap; here 0.30 Å.

3. The paper first drops the worst 5% of central carbons: r.m.s.d.$_{95}$.

$$\sqrt{(0.24^2 + 0.21^2 + 0.35^2 + 0.33^2 + 0.33^2)/5} = 0.30 \text{ Å}$$

Lower is better: 0 Å is perfect, and a carbon atom is about 1.4 Å wide.

Jumper et al., Nature 596 (2021), p. 584; Methods (Metrics); the toy: ours

5

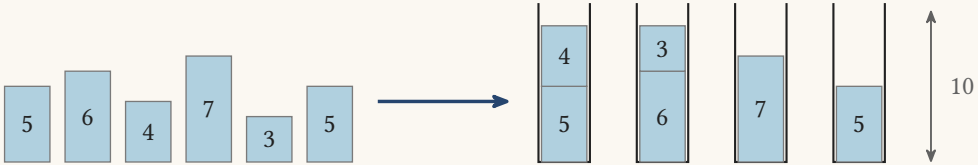
Paper 5

FunSearch

A model proposes programs, an evaluator decides

Bin packing: put every item in a bin

One of FunSearch's two examples: placing jobs, by memory size, on nodes.



1. Items: each has a size.

2. Bins: each has a capacity, here 10.

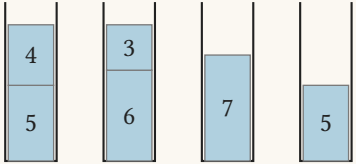
3. A packing puts every item in one bin: here, 4 bins.

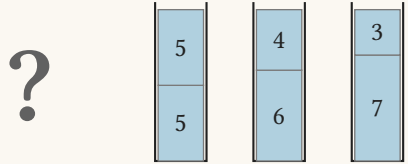
Bin packing: every item goes in a bin, and fewer bins is better.

Romera-Paredes et al., Nature 625 (2024), p. 472; the toy and the job reading: ours

Some answers are easy to check and hard to find

Checking a packing takes a moment; finding the best one does not.





1. Evaluate: count the bins; check that none holds more than 10.

2. Solve: find the fewest bins. Bin packing is NP-hard.

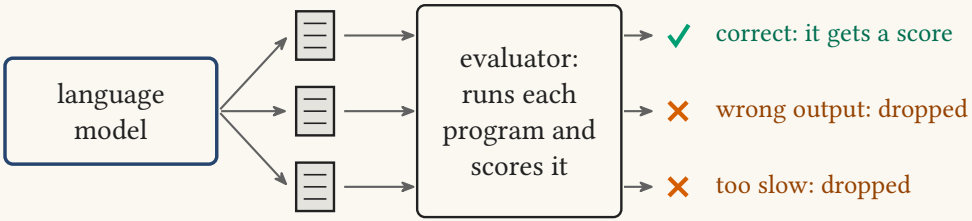
3. Trying every possible split takes too long when there are many items.

Checking is easy and finding the best is hard, so people use rules of thumb.

Nature 625 (2024), pp. 468 and 472; the toy: ours; NP-hardness: background

A model proposes, an evaluator decides

Goal: a better rule of thumb for packing. A language model writes rules as code.



1. The model writes candidate programs. Some are wrong.

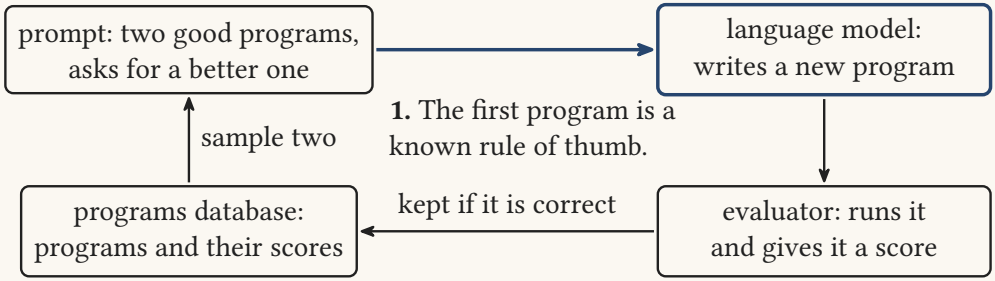
2. The evaluator tries each program on packing problems and scores it.

The evaluator drops confabulations: plausible programs that are wrong.

Romera-Paredes et al., Nature 625 (2024), pp. 468 to 470; the icons: ours

FunSearch: sample, write, score, keep

The scored programs are kept, and the best ones seed the next prompt.



This loop is FunSearch: only correct programs are kept.

Romera-Paredes et al., Nature 625 (2024), pp. 469, 470 and 472 (Fig. 1)

6

Paper 6

AlphaEvolve

Multiplying matrices with fewer multiplications

Two 2x2 matrices: eight multiplications

Multiplying matrices underpins machine learning; start with the smallest case.

$$\underbrace{\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}}_{A} \times \underbrace{\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}}_{B} = \underbrace{\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}}_{C}$$

$$\begin{aligned} 1 \cdot 5 + 2 \cdot 7 &= 19 \\ 1 \cdot 6 + 2 \cdot 8 &= 22 \\ 3 \cdot 5 + 4 \cdot 7 &= 43 \\ 3 \cdot 6 + 4 \cdot 8 &= 50 \end{aligned}$$

1. Each entry of C is a row of A times a column of B.

2. That takes two multiplications per entry.

3. Four entries: 8 multiplications and 4 additions.

The way taught at school: 8 multiplications for two 2x2 matrices.

AlphaEvolve white paper (2025), not peer reviewed, p. 9; the matrices: ours

Strassen's trick: seven products of sums

In 1969 Strassen found a way with one multiplication fewer.

$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$ the same A and B

$$
\begin{aligned}
M_1 &= (1+4)\times(5+8) &&= 65 \\
M_2 &= (3+4)\times 5 &&= 35 \\
M_3 &= 1\times(6-8) &&= -2 \\
M_4 &= 4\times(7-5) &&= 8 \\
M_5 &= (1+2)\times 8 &&= 24 \\
M_6 &= (3-1)\times(5+6) &&= 22 \\
M_7 &= (2-4)\times(7+8) &&= -30
\end{aligned}
$$

} 7 products

1. Each product: a sum of entries of A times a sum of entries of B.

2. Seven products: seven multiplications.

3. The sums cost 10 additions.

Seven multiplications, if the seven products can rebuild C.

Strassen (1969), the white paper's ref. 95: background; the values: ours

Seven products rebuild all four entries

Each entry of C is a sum of some of the seven products, with signs.

$$
\begin{aligned}
c_{11} &= M_1 + M_4 - M_5 + M_7 &&= 65 + 8 - 24 - 30 &&= 19 \checkmark \\
c_{12} &= M_3 + M_5 &&= -2 + 24 &&= 22 \checkmark \\
c_{21} &= M_2 + M_4 &&= 35 + 8 &&= 43 \checkmark \\
c_{22} &= M_1 - M_2 + M_3 + M_6 &&= 65 - 35 - 2 + 22 &&= 50 \checkmark
\end{aligned}
$$

the school way gave $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

1. Only additions and subtractions: no new multiplication.

2. The same C as the school way.

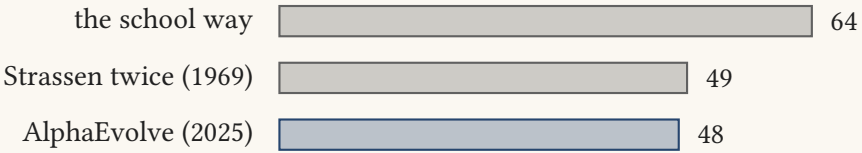
3. Our script checks all pairs of 2x2 matrices.

7 multiplications instead of 8, paid for with 18 additions instead of 4.

Strassen (1969): background; values and check: ours (numbers/alphaevolve_numbers.py)

For 4x4 complex matrices: 48

AlphaEvolve, a 2025 white paper of Google DeepMind, reports a recipe with 48.



1. For 4x4 matrices: 64 the school way, 49 with Strassen's recipe twice.

2. AlphaEvolve: 48, for complex matrices.

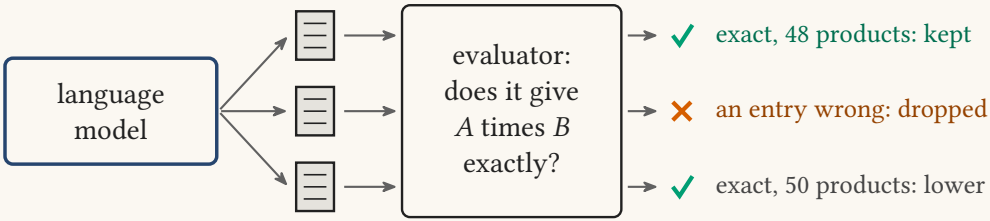
3. The paper: the first gain over Strassen "after 56 years", "in this setting".

One multiplication fewer than a record from 1969.

AlphaEvolve white paper (2025), not peer reviewed, pp. 1, 3 and 10

A recipe can be checked by a program

As in FunSearch, the previous paper: a model proposes, a program checks.



1. A candidate recipe is a list of products and sums.

2. The check is mechanical, and fewer products score higher.

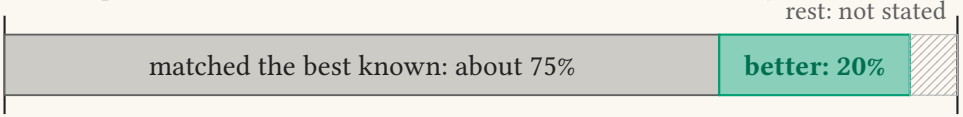
Checking a recipe needs no mathematician: a program decides.

AlphaEvolve white paper (2025), not peer reviewed, pp. 2, 3 and 10; the drawing: ours

Over 50 open problems: about 20% improved

In many open problems, one better example proves a better bound.

over 50 problems, in more than five branches of mathematics, at many sizes



1. Open problems proposed by mathematicians, among them Terence Tao.

2. Each run began from a simple or random example.

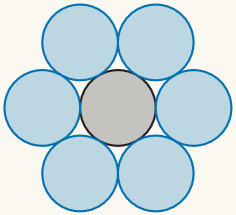
3. It matched about 75%, and beat about 20%.

In about 20% of the problems, a new and provably better construction.

AlphaEvolve white paper (2025), not peer reviewed, pp. 3, 10 and 13

Kissing number: 593 spheres in 11 dimensions

How many equal balls can touch one ball without overlapping each other?



1. In the plane, six coins touch one; a seventh has no room.

in 11 dimensions

best known → AlphaEvolve

592 → **593**

2. The same question in d dimensions: the kissing number.

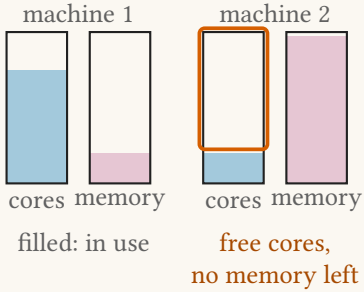
3. In 11 dimensions the record was 592.

One sphere more than the record: a better example, so a better bound.

AlphaEvolve white paper (2025), not peer reviewed, pp. 13, 42, 43; the plane: background, ours

Scheduling: 0.7% of Google's compute back

Placing jobs on machines is bin packing again, now with two sizes: cores and memory.



evolved rule: gives each machine a priority

Google's fleet: 0.7% of compute no longer stranded

1. Out of memory, a machine strands its free cores.

2. The rule is short code, as in FunSearch.

3. Simulated first, then deployed.

A short evolved rule, readable, and now in use across Google's fleet.

AlphaEvolve white paper (2025), not peer reviewed, pp. 13, 14 (Fig. 6); the machines: ours